

Comprehensive University Project Report: Smart Lost & Found Management System

SEUSL Group Project

Team Members

- **Nisala** (GitHub: [@nisalajuriance](https://github.com/nisalajuriance) (<https://github.com/nisalajuriance>))
- **Dinuka** (GitHub: [@dinukahansana](https://github.com/dinukahansana) (<https://github.com/dinukahansana>))
- **Heshan** (GitHub: [@Heshan-3](https://github.com/Heshan-3) (<https://github.com/Heshan-3>))
- **Janidu** (GitHub: [@janidu200301](https://github.com/janidu200301) (<https://github.com/janidu200301>))

1. Executive Summary

The **Smart Lost & Found Management System** is a full-stack, AI-integrated web application built to streamline the reporting, tracking, and recovery of lost and found items. Designed specifically for large organizational ecosystems like universities, the platform integrates robust user authentication, real-time notifications, intelligent item matching, and a highly secure peer-to-peer verification workflow.

2. Problem Statement & Objectives

2.1 The Problem

Traditional lost and found operations rely on physical notice boards, unsecured social media groups, or centralized administrative offices. This disjointed approach leads to:

1. **Poor Visibility:** Lost items go unnoticed by those who found them.
2. **Privacy Risks:** Publicly sharing phone numbers on WhatsApp groups leads to spam and harassment.
3. **Verification Fraud:** Malicious users claiming high-value items (laptops, phones) that do not belong to them.
4. **Inefficient Matching:** No automated way to link a lost item report with a found item report.

2.2 Objectives

1. Digitize the entire lifecycle of a lost/found item.
2. Automate the matching of lost and found items using AI and smart algorithms.
3. Protect user privacy until verification is necessary.
4. Provide a secure mechanism for evidence submission and peer-to-peer handover.

3. Technology Stack & Architecture

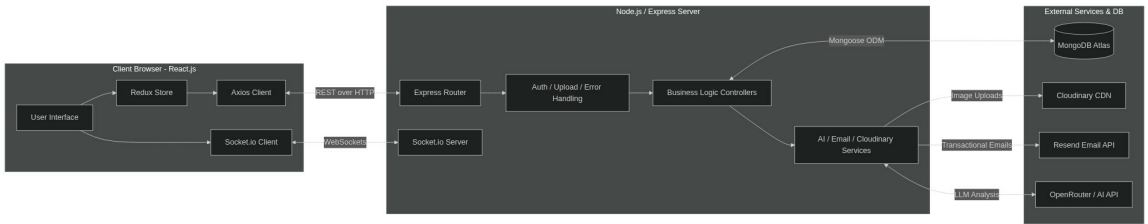
3.1 Stack Overview (MERN + AI)

The project is built on the **MERN** stack, enhanced with AI services and real-time WebSockets.

Layer	Technology	Purpose
	React 18, Vite, Tailwind CSS,	Provides a modern, responsive, glassmorphism UI with

Frontend	Framer Motion	smooth animations.
State Management	Redux Toolkit (<code>@reduxjs/toolkit</code>)	Centralized global state for user sessions, items, and UI states.
Backend	Node.js, Express.js	High-performance REST API routing and business logic execution.
Database	MongoDB Atlas (Mongoose)	Flexible, scalable document database for dynamic schema requirements.
Real-time	Socket.io	Bidirectional communication for instant push notifications and alerts.
Storage	Cloudinary	Secure cloud storage and on-the-fly optimization for user-uploaded images.
Communication	Resend API / Nodemailer	Transactional emails (OTPs, match alerts, claim approvals).
AI Integration	Opencode.ai / OpenRouter	Intelligent categorization and automated matching of lost & found descriptions.

3.2 System Architecture Diagram



4. Database Schema Design

The MongoDB database uses highly normalized references to maintain data integrity.

4.1 User Schema (`User.js`)

- `fullName`, `email`, `studentId`, `password` (hashed).
- `role`: Enum (`user`, `admin`).
- `status`: Enum (`active`, `suspended`).
- Handles password reset tokens and verification statuses.

4.2 Item Schemas (`LostItem.js` & `FoundItem.js`)

- `itemName`, `category` (ObjectId ref to Category), `description`.
- `dateLost` / `dateFound`, `location`, `time`.
- `images`: Array of objects containing `url` and `public_id` from Cloudinary.
- `status`: Enum (`available`, `pending`, `matched`, `claimed`).
- `userId`: ObjectId reference to the user who reported it.

4.3 Claim Request Schema (`ClaimRequest.js`)

- `foundItemId` / `lostItemId`: References to the item being claimed.

- `claimantId`: Reference to the user making the claim.
- `proofDescription`: Text explanation proving ownership.
- `proofImages`: Supporting image evidence.
- `status`: Enum (`pending`, `approved`, `rejected`).
- `isContactShared`: **Boolean**. Essential for the privacy-first claiming workflow.

4.4 Additional Models

- **Notification.js**: Stores user-specific alerts (`title`, `message`, `read` status).
 - **Match.js**: Stores AI-generated match scores between Lost and Found items.
 - **AdminLog.js**: Audit trails for administrative actions.
 - **Feedback.js**: User feedback collection.
-

5. Core Workflows & Logic

5.1 Smart AI Item Matching

When a user submits a lost item, the system doesn't just save it.

1. The backend triggers `matchController.js`.
2. It queries recently found items in the same timeframe and category.
3. The descriptions and image tags are sent to the **AI Service** (`aiController.js`) via OpenRouter API.
4. The AI returns a **Match Score** (percentage).
5. If the score exceeds a predefined threshold (e.g., 75%), a `Match` record is created, and both users are notified instantly via Socket.io.

5.2 The "Pre-Approval Contact Sharing" Workflow (Privacy First)

A major challenge in Lost & Found systems is verifying ownership without exposing the finder's personal details to scammers. We engineered a multi-tier verification flow:

1. **Claim Submission**: User A recognizes their lost phone in the "Found Items" feed. User A submits a claim with proof (e.g., IMEI screenshot).
2. **Review Stage**: User B (the finder) receives a notification. User B reviews the proof.
3. **Contact Sharing (Crucial Step)**: Instead of clicking "Approve" (which permanently finalizes the transaction), User B clicks **"Share My Contact"**.
 - The `isContactShared` flag becomes `true`.
 - User A receives an email via **Resend API** with User B's phone number.
4. **Peer-to-Peer Verification**: User A calls User B. User B asks verification questions (e.g., "What is the lock screen wallpaper?").
5. **Final Confirmation**: Upon successful verbal verification, User B clicks **"Confirm as True Owner"**. The system then officially marks the item as `claimed` and auto-rejects any other pending claims.

5.3 Secure & Privacy-First Image Handling

1. **Frontend Compression & EXIF Stripping**: Using `browser-image-compression`, images are not only compressed to save bandwidth, but all **EXIF metadata (including GPS coordinates)** is stripped client-side before upload. This ensures that malicious actors cannot extract the exact location or time a photo was taken, protecting student privacy.
2. **Backend Upload**: Express receives the sanitized file via `multer`.

3. **Cloud Storage:** The file is streamed to **Cloudinary**, which stores it securely and returns an optimized CDN URL.

5.4 Frictionless Onboarding (Google OAuth & Auto-Login)

To reduce friction during onboarding, the platform uses two strategies:

1. **Auto-Login:** Users who register manually via email/password are automatically logged in upon success, removing the redundant step of re-entering credentials.
2. **Google Sign-In & Profile Completion:** Users can authenticate via Google. Because a typical Google account lacks specific university metadata (e.g., Student ID), a custom Profile Completion flow was implemented. If `studentId` or `phone` is missing, a specialized `incomplete_profile` JWT token is issued. The React frontend intercepts this and forces a **Profile Completion Modal**. Users can optionally upload a profile picture during this step (which also undergoes EXIF stripping) to help peers identify them during physical item handovers.

5.5 Dynamic System Configuration & Admin Dashboard

An admin-facing **Dashboard** allows non-technical administrators to control core system behaviors without code deployments:

- **System Metrics & User Management:** Admins can view platform statistics and manually suspend or reinstate users.
- **Email Verification Toggle:** Admins can enforce or bypass mandatory email verification during registration (useful for rapid onboarding during university orientations).
- **Contact Info Management:** Dynamic updates to the support email, phone lines, and physical office location for the public "Contact Us" page.
- **Fraud Limits:** Dynamic adjustment of spam thresholds (e.g., max pending claims, max rejected claims).

6. Security Integrations & Anti-Fraud Algorithms

Security is paramount, especially when handling student data and physical item handovers.

- **Data Protection:** `helmet` is used to secure HTTP headers.
- **DDoS Prevention:** `express-rate-limit` combined with `rate-limit-redis` restricts the number of API calls a user can make within a time window.
- **NoSQL Injection Guard:** `express-mongo-sanitize` strips out malicious `$`, `{`, and `.` characters from incoming request bodies.
- **Authentication:** Sessions are completely stateless. JWTs (Access and Refresh tokens) are issued upon login and validated in a custom `protect` middleware on every secure route.

6.1 Advanced Anti-Fraud & Velocity Checks (AI-Behavior Simulation)

To prevent malicious users from spamming the system or attempting to steal items via rapid claim submissions, a multi-layered behavioral defense mechanism is built into the `claimController`:

1. **Concurrent Claim Limit:** Users are restricted to a maximum number of active (pending) claims at any given time (default: 5).
2. **Velocity Check:** The system analyzes the user's submission history. If a user submits more claims than physically plausible within a 24-hour rolling window (default: 3 claims/day), it is flagged as highly anomalous.
3. **Rejection Thresholds:** If a user accumulates a specific number of rejected claims (default: 3), indicating repeated failed attempts to prove ownership.
4. **Automated Enforcement:** Triggering either the Velocity Check or the Rejection Threshold results in an **Immediate Auto-Ban**. The user's `isActive` flag is instantly revoked, their session is destroyed, and an

automated "Account Suspended" email is dispatched to their university email address.

7. Frontend Architecture Highlights

- **Redux Slices:** Data is segmented logically (e.g., `authSlice`, `itemSlice`, `claimSlice`). Async Thunks handle all API communications.
 - **Reusable Components:** UI elements like `Button`, `Modal`, `StatusBadge`, and `ItemCard` are highly modularized.
 - **Responsive Design:** Built Mobile-First using Tailwind CSS, ensuring the dashboard looks flawless on smartphones and desktop monitors alike.
 - **Glassmorphism Aesthetic:** The UI uses backdrop-blur, semi-transparent panels, and neon accents to provide a premium, modern feel.
-

8. Production-Grade System Optimizations

To ensure the system remains highly available, responsive, and secure under load, the following enterprise-level optimizations have been implemented:

1. **API Security & Hardening:** The backend is fortified with `helmet` for secure HTTP headers, `express-mongo-sanitize` to prevent NoSQL query injections, and a robust `express-rate-limit` configuration (capped at 1000 requests per 15 mins) powered by Redis to mitigate DDoS and brute-force attacks.
 2. **Database Performance Indexing:** MongoDB schemas have been heavily optimized. Compound and single-field indexes (e.g., `similarityScore`, `createdAt`, `{ lostItemId: 1, foundItemId: 1 }`) are applied to the `Match` and `Item` collections, enabling ultra-fast query resolution even with massive datasets.
 3. **Real-time WebSockets:** A high-performance `socket.io` architecture handles instantaneous bidirectional communication between the server and clients. Notifications for AI matches and claim updates are pushed in real-time without requiring manual page refreshes.
-

9. Conclusion

The **Smart Lost & Found Management System** goes beyond simple CRUD operations. By integrating AI for smart matching, employing a privacy-first contact sharing workflow, enforcing robust security and performance optimizations, and utilizing modern cloud-native tools (Cloudinary, Resend, MongoDB Atlas, Redis), it provides a production-ready solution to a widespread organizational problem. The architecture is highly decoupled, ensuring that future enhancements (such as mobile app integration or advanced image recognition) can be added seamlessly.